

How the medium sort works, step by step

Chunk sort (k-sort distribution, then cheapest insertion back): algorithms/medium.c, reusing simple_cost.c and simple_rotation.c

The idea: move every number from A to B in overlapping chunks of nearby ranks, so B ends up *roughly* sorted, with big numbers near the top. Then bring them back with the same cheapest-insertion pass as the simple sort, which keeps A sorted ascending. Inputs of 5 numbers or fewer use `fixed_sort` instead.

1 Window

Pick the window size: $\text{delta} = \text{ft_sqrt}(\text{size}) \times 1.5$, rounded down. For 8 numbers: $2 \times 1.5 = 3$.

`calculate_delta`

2 Distribute

Look at the top of A. If its rank is inside the window, pb (and rb if it is in the low part); otherwise ra. A is emptied.

`apply_k_distribution`

3 Back to A

While B is not empty: push the cheapest into A, on top of its next-bigger number.

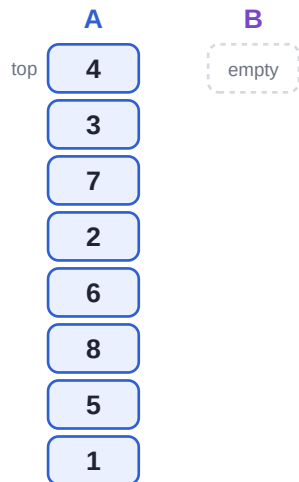
`get_cheapest_move(to_b = 0)`

4 Align

A is sorted but maybe rotated. Rotate its minimum to the top the short way.

`final_alignment`

THE EXAMPLE IN THIS WALKTHROUGH



Input `./push_swap --medium 4 3 7 2 6 8 5 1`. The first argument ends up on top of A.

The code compares each node's index (its rank), not its value. With the numbers 1 to 8 the rank is just value minus 1, so the pages show values; when a rule talks about ranks, it says so.

In total the algorithm needs **30 operations**. Every step on the following pages was produced by a Python replica of the C code whose output was checked against the real `push_swap` binary.

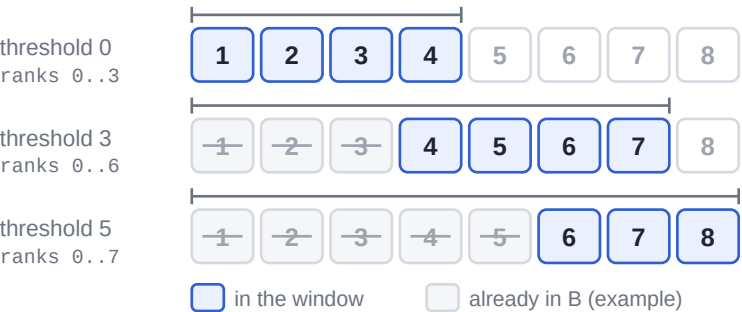
Colours: an **orange** box is the number chosen to move, a **teal** box is its target (the number it will land on).

The two rules behind distribution

Which number gets pushed, and where in B it lands.

1. A window that slides up

threshold starts at 0 and goes up by 1 after every push. The top of A is pushed when its rank is $\leq \text{threshold} + \text{delta}$; otherwise ra brings the next number up. With 8 numbers, $\text{delta} = \text{ft_sqrt}(8) \times 1.5 = 2 \times 1.5 = 3$.



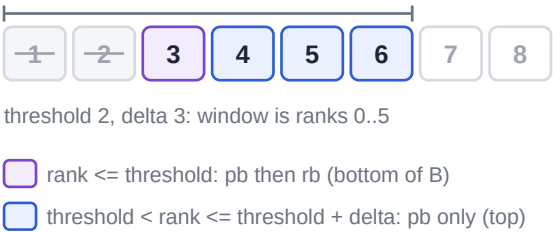
Why $\leq$ and not $<$? The inclusive bound makes the window hold $\text{delta} + 1$ ranks. After k pushes the window is ranks $0..k+\text{delta}$: $k + \text{delta} + 1$ ranks, of which at most k have left A. So at least one is always still in A and the ra loop is guaranteed to find it, even if delta were 0. It also lines up with the $\leq \text{threshold}$ test in rule 2.

window size by input

size	ft_sqrt(size)	delta = x 1.5
8	2	3
100	10	15
500	22	33

2. The low part goes to the bottom of B

Right after pb, if B holds at least 2 numbers and the new top has rank $\leq \text{threshold}$, rb sends it to the bottom of B. Ranks above threshold stay on top.



The goal is a rough order, not a sorted B: numbers of similar rank tend to arrive close together, with the higher part of each chunk on top and the lower part underneath. Phase 3 does not rely on that order; it prices every number anyway, it just finds cheaper moves when B is close to sorted.

The `ft_lstsize(stacks->b) >= 2` check skips the rotate when B has only one number: rb on it would change nothing and waste an operation.

Phase 3 reuses the simple sort's costing

Back to A is exactly `push_cheapest(stacks, 0, print)`: a signed rotation cost per stack (top half ra/rb, bottom half rra/rrb), and shared rr/rrr when both go the same way. The simple-sort walkthrough explains those two rules in detail.

Distribute: steps 1 to 5 of 9

window, stays on top

window, low part

already in B

top of A

Each card: the window before the step, the decision, the operations, then A and B after it.

Step 1

threshold 0

1

2

3

4

5

6

7

8

Top of A is 4 (rank 3). $3 \leq 3$, so push it. B had nothing, so no rb.

pb

A

3

7

2

6

8

5

1

B

4

Step 2

threshold 1

1

2

3

4

5

6

7

8

Top of A is 3 (rank 2). $2 \leq 4$, so push it. Rank 2 > threshold 1: it stays on top.

pb

A

7

2

6

8

5

1

B

3

4

Step 3

threshold 2

1

2

3

4

5

6

7

8

Top of A is 7 (rank 6). $6 > 5$: outside the window, so ra and look again.

ra

A

2

6

8

5

1

7

B

3

4

Step 4

threshold 2

1

2

3

4

5

6

7

8

Top of A is 2 (rank 1). $1 \leq 5$, so push it. Rank 1 <= threshold 2: rb to the bottom.

pb rb

A

6

8

5

1

7

B

3

4

2

Step 5

threshold 3

1

2

3

4

5

6

7

8

Top of A is 6 (rank 5). $5 \leq 6$, so push it. Rank 5 > threshold 3: it stays on top.

pb

A

8

5

1

7

B

6

3

4

2

Distribute: steps 6 to 9 of 9

 window, stays on top window, low part already in B top of A

Each card: the window before the step, the decision, the operations, then A and B after it.

Step 6

threshold 4

1

2

3

4

5

6

7

8

Top of A is **8** (rank 7). $7 \leq 7$, so push it. Rank 7 > threshold 4: it stays on top.

pb

A

B

5

1

7

8

6

3

4

2

Step 7

threshold 5

1

2

3

4

5

6

7

8

Top of A is **5** (rank 4). $4 \leq 8$, so push it. Rank 4 $\leq$ threshold 5: rb to the bottom.

pb rb

A

B

1

7

8

6

3

4

2

5

Step 8

threshold 6

1

2

3

4

5

6

7

8

Top of A is **1** (rank 0). $0 \leq 9$, so push it. Rank 0 $\leq$ threshold 6: rb to the bottom.

pb rb

A

B

7

8

6

3

4

2

5

1

Step 9

threshold 7

1

2

3

4

5

6

7

8

Top of A is **7** (rank 6). $6 \leq 10$, so push it. Rank 6 $\leq$ threshold 7: rb to the bottom.

pb rb

A

B

empty

8

6

3

4

2

5

1

7

Where B ended up

A is empty. Phase total: **13 ops** (8 pb, 4 rb, 1 ra).

B, top to bottom: **8 6 3 4 2 5 1 7**.

The big 8 and 6 stayed on top. Steps 7 to 9 each pushed a rank at or below threshold, so each got rb and 5 1 7 piled up at the bottom in arrival order.

With only 8 numbers the window (4 ranks) is half the input, so B is loose. With 100 or 500 numbers each chunk is a much smaller slice of the whole.

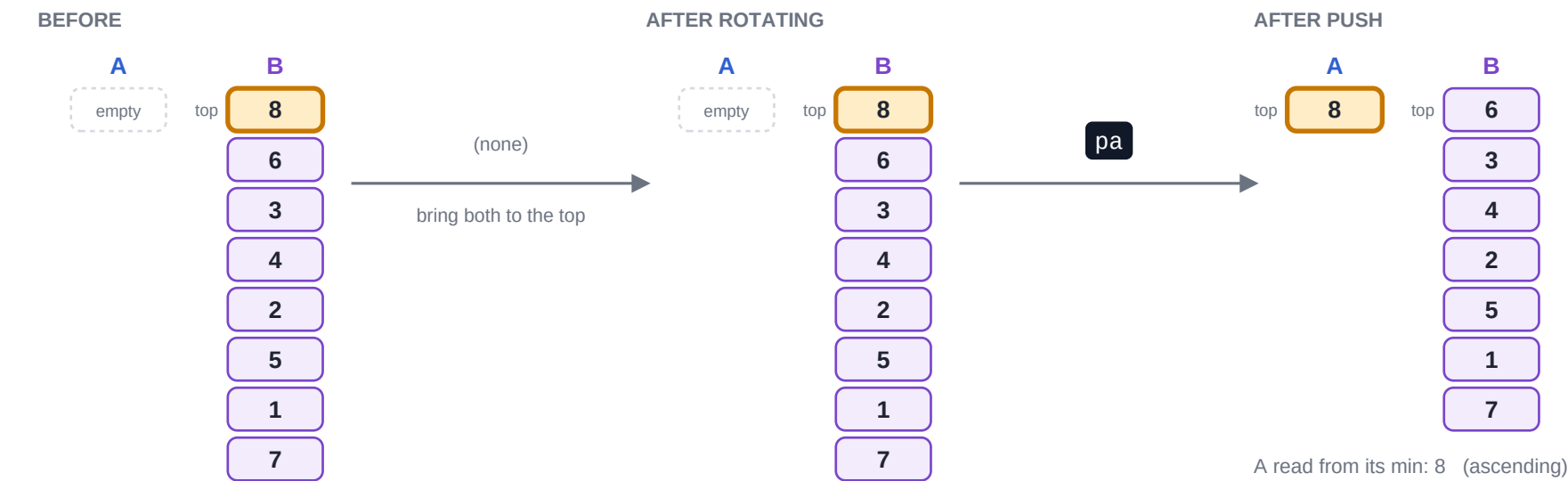
Notice the last pushes: once threshold + delta reaches the top rank, every remaining number is in the window, so A empties without any more ra.

Push 1 of 8: move 8 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
8	none	0	0	0
6	none	0	+1 (1 rb)	1
3	none	0	+2 (2 rb)	2
4	none	0	+3 (3 rb)	3
2	none	0	+4 (4 rb)	4
5	none	0	-3 (3 rrb)	3
1	none	0	-2 (2 rrb)	2
7	none	0	-1 (1 rrb)	1

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 8 (total 0; the first lowest total wins ties). A is still empty, so there is no target at all: `get_larger_target` returns NULL and cost A is 0. On this push every candidate only pays for its own rotation in B, so the top of B wins.

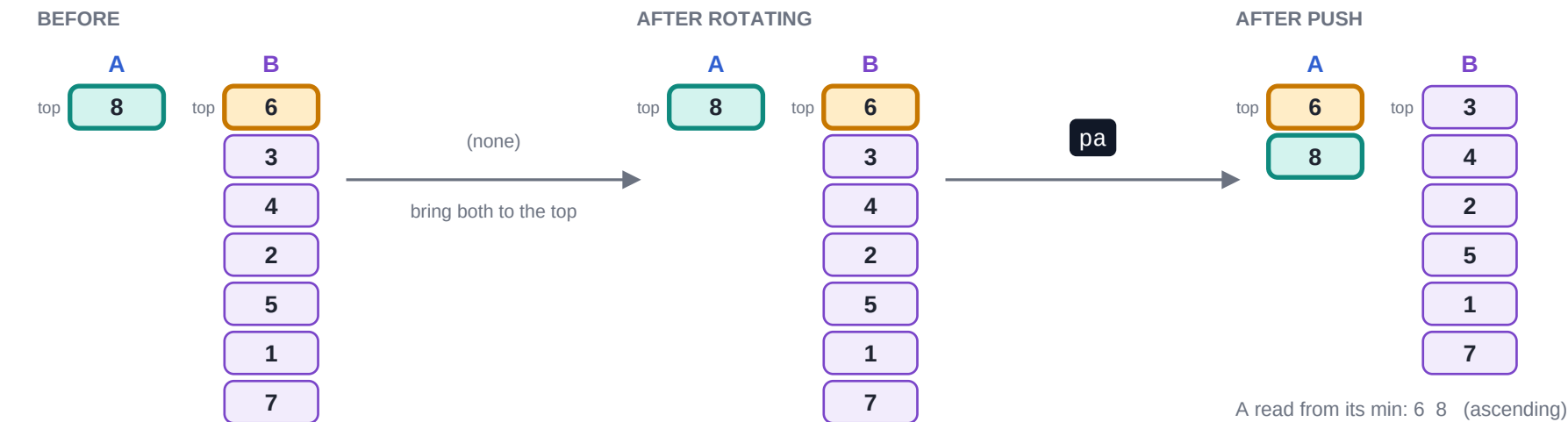
Moves: A is empty and 8 is already on top of B, so no rotations: `pa` simply moves 8 into A.

Push 2 of 8: move 6 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
6	8	0	0	0
3	8	0	+1 (1 rb)	1
4	8	0	+2 (2 rb)	2
2	8	0	+3 (3 rb)	3
5	8	0	-3 (3 rrb)	3
1	8	0	-2 (2 rrb)	2
7	8	0	-1 (1 rrb)	1

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 6 (total 0; the first lowest total wins ties). The smallest number in A above 6 is **8**.

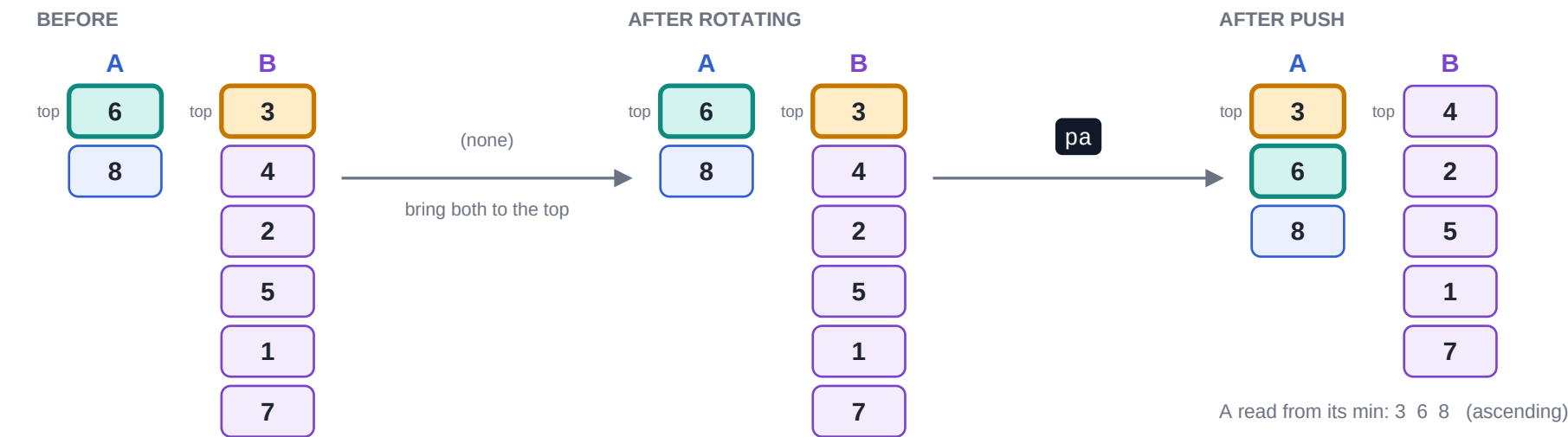
Moves: 6 and 8 are both already on top, so no rotations: pa drops 6 straight onto 8.

Push 3 of 8: move 3 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
3	6	0	0	0
4	6	0	+1 (1 rb)	1
2	6	0	+2 (2 rb)	2
5	6	0	+3 (3 rb)	3
1	6	0	-2 (2 rrb)	2
7	8	+1 (1 ra)	-1 (1 rrb)	2

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 3 (total 0; the first lowest total wins ties). The smallest number in A above 3 is **6**.

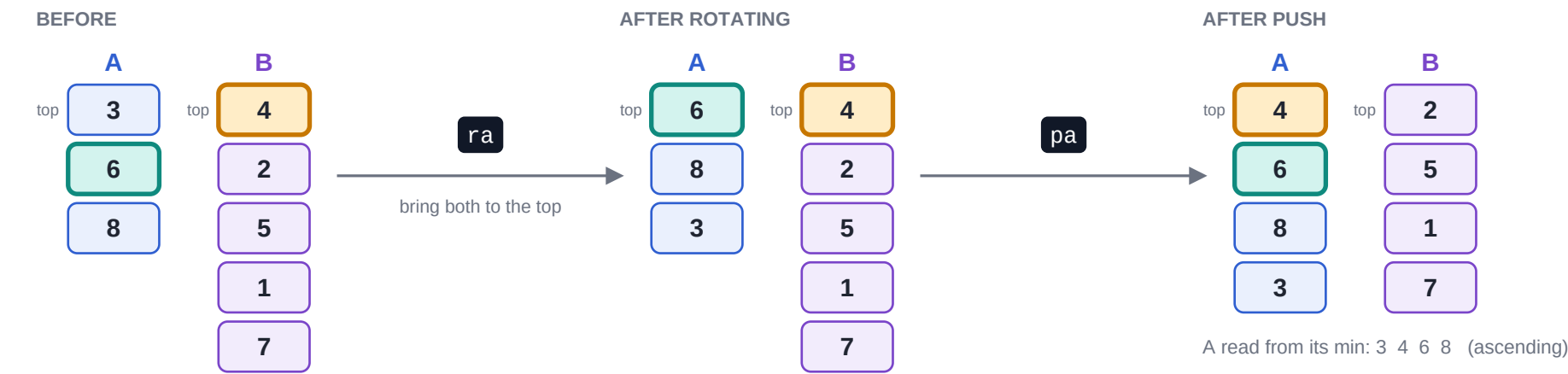
Moves: 3 and 6 are both already on top, so no rotations: pa drops 3 straight onto 6.

Push 4 of 8: move 4 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
4	6	+1 (1 ra)	0	1
2	3	0	+1 (1 rb)	1
5	6	+1 (1 ra)	+2 (2 rb)	2
1	3	0	-2 (2 rrb)	2
7	8	-1 (1 rra)	-1 (1 rrb)	1

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 4 (total 1; the first lowest total wins ties). The smallest number in A above 4 is **6**.

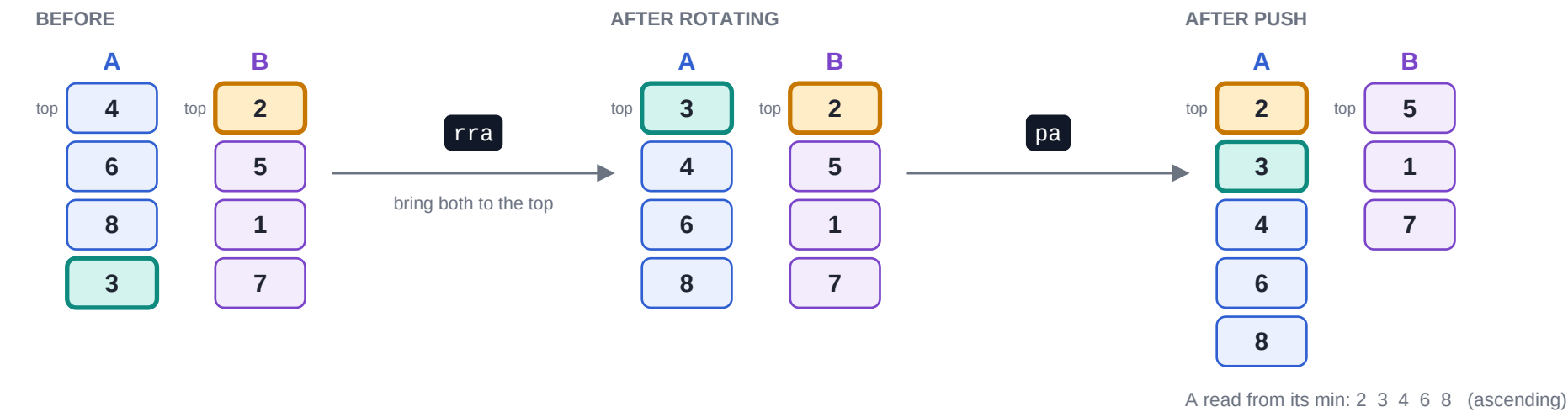
Moves: Only one stack needs to move: total = 1. Run 1x ra, then pa drops 4 on top of 6.

Push 5 of 8: move 2 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
2	3	-1 (1 rra)	0	1
5	6	+1 (1 ra)	+1 (1 rb)	1
1	3	-1 (1 rra)	+2 (2 rb)	3
7	8	+2 (2 ra)	-1 (1 rrb)	3

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 2 (total 1; the first lowest total wins ties). The smallest number in A above 2 is **3**.

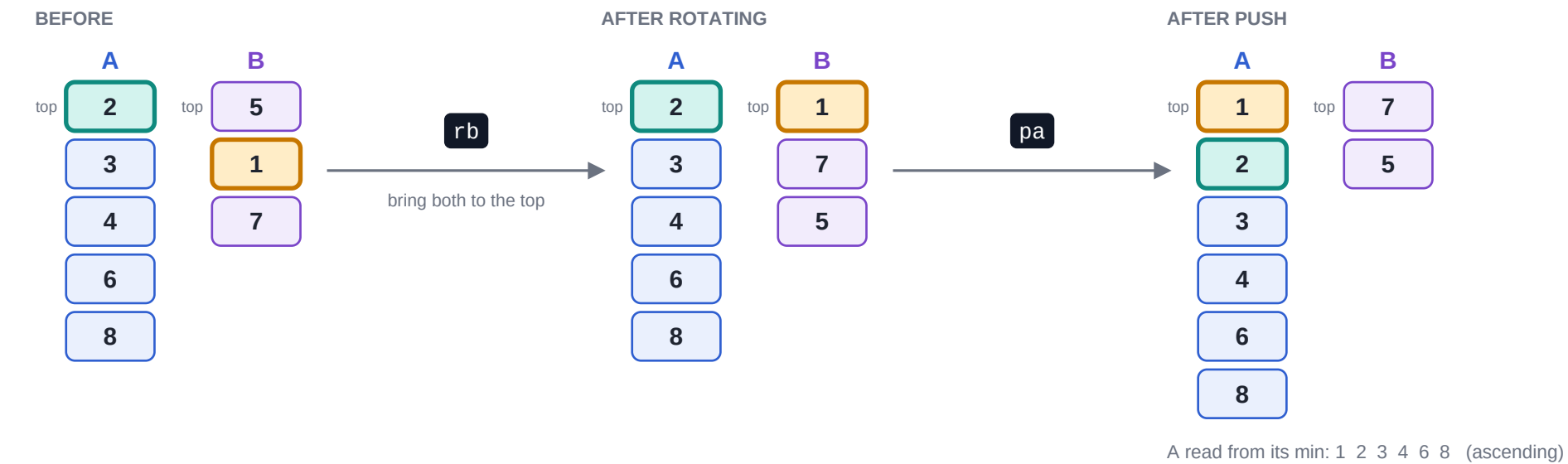
Moves: Only one stack needs to move: total = 1. Run 1x rra, then pa drops 2 on top of 3.

Push 6 of 8: move 1 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
5	6	-2 (2 rra)	0	2
1	2	0	+1 (1 rb)	1
7	8	-1 (1 rra)	-1 (1 rrb)	1

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 1 (total 1; the first lowest total wins ties). The smallest number in A above 1 is **2**.

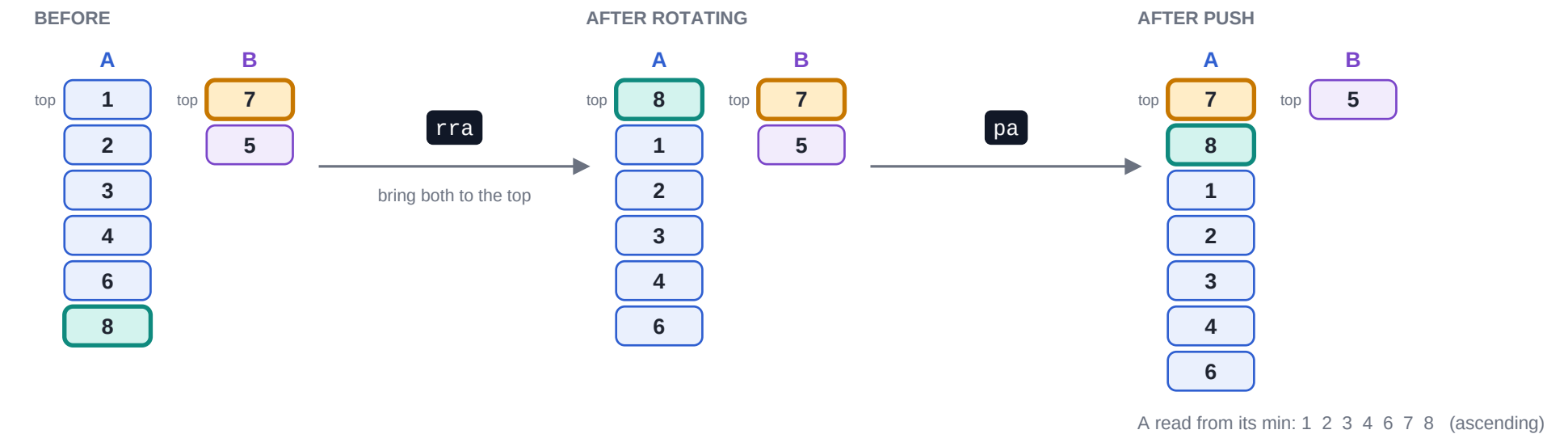
Moves: Only one stack needs to move: total = 1. Run 1x rb, then pa drops 1 on top of 2.

Push 7 of 8: move 7 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
7	8	-1 (1 rra)	0	1
5	6	-2 (2 rra)	+1 (1 rb)	3

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 7 (total 1; the first lowest total wins ties). The smallest number in A above 7 is 8.

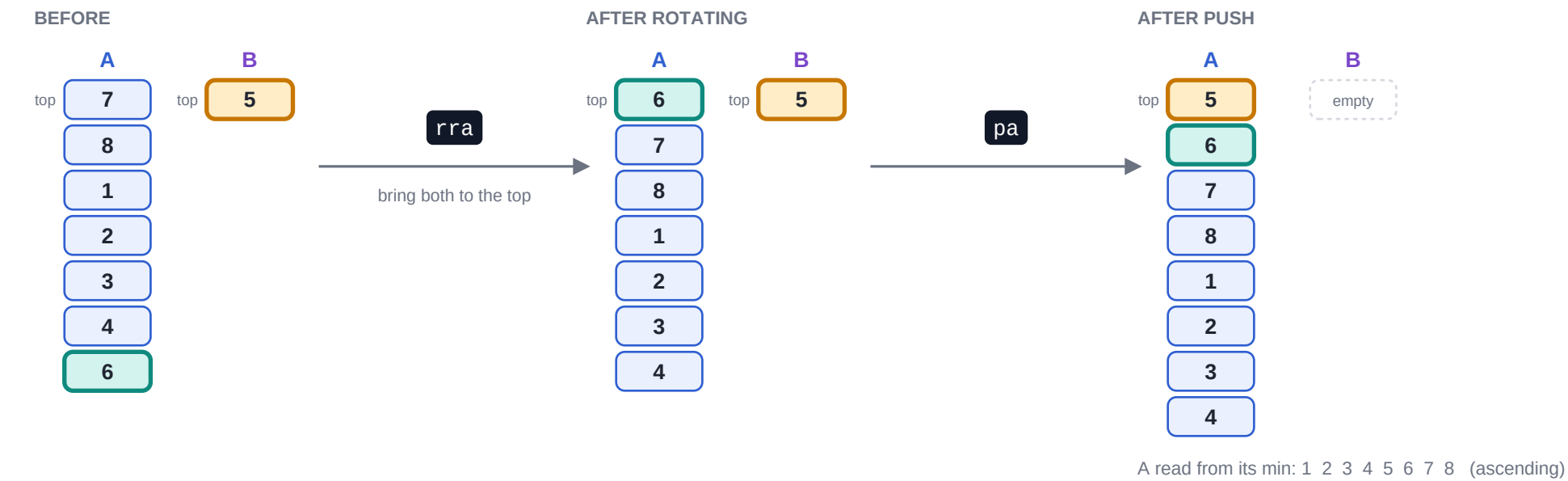
Moves: Only one stack needs to move: total = 1. Run 1x rra, then pa drops 7 on top of 8.

Push 8 of 8: move 5 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.

chosen number

its target



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
5	6	-1 (1 rra)	0	1

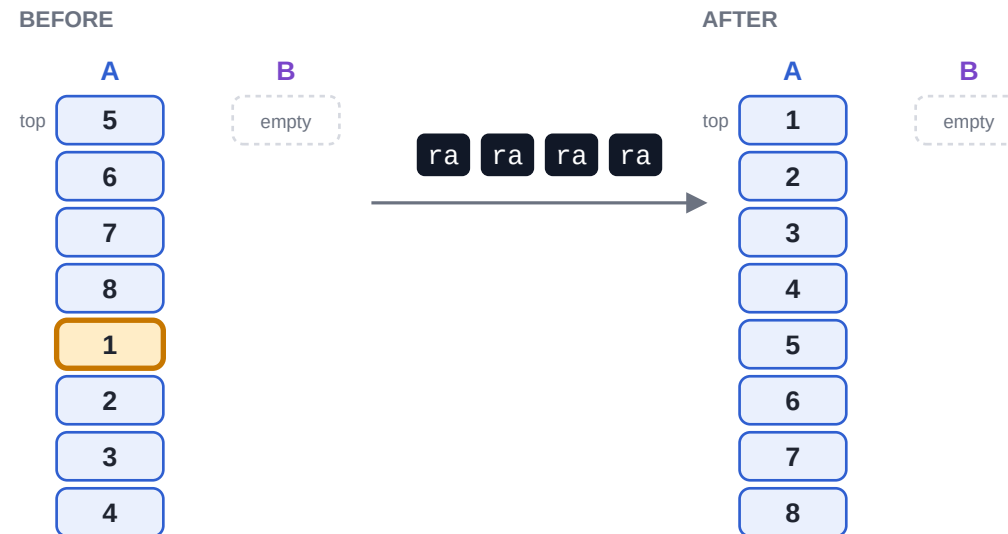
Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 5 (total 1; the first lowest total wins ties). The smallest number in A above 5 is **6**.

Moves: Only one stack needs to move: total = 1. Run 1x rra, then pa drops 5 on top of 6.

Rotate the minimum to the top

A is sorted in a circle but may start in the wrong place.



A reads 1 2 3 4 5 6 7 8 from its minimum, so it is sorted but rotated. The minimum 1 is at position 4 of 8.

`final_alignment` reuses the same signed cost: position 4 $\leq$ size / 2 = 4, so the cost is **+4**, which means 4x ra.

With 8 numbers, position 4 is exactly halfway: 4x rra would cost the same. The $\leq$ in `get_rotation_cost` breaks the tie towards ra.

Done: sorted in 30 operations

The full operation list, exactly as `./push_swap --medium` prints it.

Distribute

13 ops

pb pb ra pb rb pb pb pb rb pb rb pb rb

Back to A

13 ops

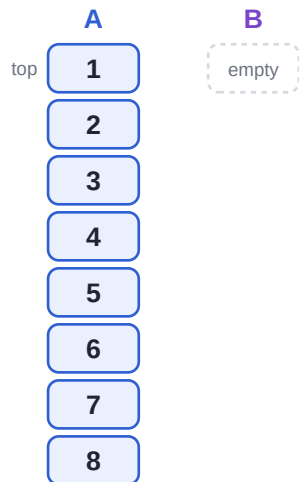
pa pa pa ra pa rra pa rb pa rra pa rra pa

Align

4 ops

ra ra ra ra

FINAL STACKS



Why 30 here when the simple sort needed 16? On 8 numbers the distribution costs 13 operations just to empty A, and B comes out only loosely ordered. The pre-sorting only pays for itself on larger inputs.

On bigger inputs (average of 20 random runs each, measured with the real binary): 100 numbers: medium **613**, simple 570 operations. 500 numbers: medium **4778**, simple 5099 operations. So in its current form medium is still behind at 100 and only pulls ahead at 500.

Why delta grows with $\sqrt{n}$? A wider window means fewer ra while hunting for the next number, but a looser B. $\sqrt{n}$ chunks of about $\sqrt{n}$ numbers balance the two, which is where the $O(n\sqrt{n})$ label in `sort.c` comes from.

Cost of the search: the distribution itself is cheap, but phase 3 still prices every number in B and walks A to find its target, so that part does $O(n^2)$ work, the same as the simple sort.